Attacking Web Applications with Ffuf

HTB Academy Module Notes · Directory, File, Vhost, and Parameter Fuzzing

---

These notes are built from the module text you shared — Section 1 (Introduction), Section 2 (Web Fuzzing), Section 3 (Directory Fuzzing), Section 8 (Vhost Fuzzing), and Section 9 (Filtering Results) so far. Sections 4–7 (likely covering file/extension fuzzing and sub-domain fuzzing, based on the module's own topic list and Section 8's references back to them) haven't been sent yet, so there's a gap in the sequence below — paste them in whenever you have them and they'll slot into place.

Module Overview

This module centres on **ffuf** (Fuzz Faster U Fool), one of the most common and reliable tools for web fuzzing — the practice of automatically testing a web application's components against a wordlist to discover what actually exists on the server.

#	Section	Status
1	Introduction	Covered below
2	Web Fuzzing	Covered below
3	Directory Fuzzing	Covered below
4–7	(likely file/extension & sub-domain fuzzing)	Not yet provided
8	Vhost Fuzzing	Covered below
9	Filtering Results	Covered below
10–13	Remaining module sections	Not yet provided

The module description lists these topics to be covered across its 13 sections:

- Fuzzing for directories
- Fuzzing for files and extensions
- Identifying hidden vhosts
- Fuzzing for PHP parameters
- Fuzzing for parameter values

Section 1 — Introduction

The module focuses on **ffuf** for directory and parameter fuzzing/brute-forcing against web applications. While many tools and methods exist for this kind of work, ffuf is highlighted as one of the most common and reliable choices.

What fuzzing actually does

A tool like ffuf automates testing a web application's individual components — for example, it works through a wordlist, sending a request to the webserver for each entry to check whether a page with that name exists. A `200` response code indicates the page exists, at which point it can be examined manually.

```
Wordlist entry → HTTP request → Response code 200 → Page exists, inspect manually
```

Section 2 — Web Fuzzing

The module's first hands-on exercise presents a target website at `http://SERVER_IP:PORT` that has no links to anything else and no information pointing to further pages. With no visible navigation, fuzzing the website becomes the only practical way forward.

What fuzzing means here

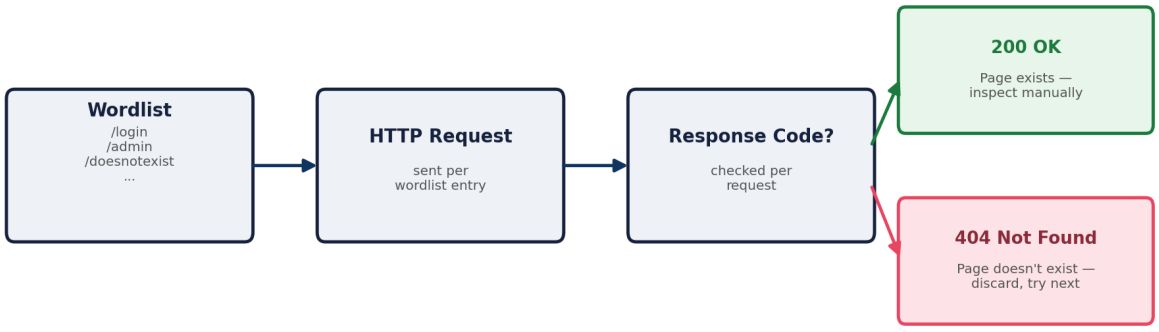
Fuzzing is a testing technique that sends various types of input to an interface to study how it reacts. The specific input depends on what's being tested for — fuzzing for SQL injection means sending random special characters and watching the server's reaction; fuzzing for a buffer overflow means sending progressively longer strings to see if and when a binary breaks.

For web fuzzing specifically, a pre-defined wordlist of commonly used terms is sent to the server to see which ones it accepts. This is necessary because web servers don't normally publish a directory of all their available links (unless badly misconfigured) — so each possible link has to be checked individually to see which ones actually return pages.

Reading the response code

Visiting a page that doesn't exist — for example `https://www.hackthebox.eu/doesnotexist` — returns an HTTP **`404`** ("Page Not Found"). Visiting a page that does exist — for example `https://www.hackthebox.eu/login` — returns the actual page content with an HTTP **`200`** ("OK"). This distinction is the entire basis of fuzzing for pages and directories.

The Fuzzing Loop: Wordlist → Request → Decision



A fuzzing tool repeats this loop hundreds of times per second across the entire wordlist

Doing this manually, one URL at a time, would take forever — which is exactly why automated tools exist. They send hundreds of requests per second, inspect each response code, and determine whether a page exists, letting the tester quickly narrow down what's actually there before examining it by hand.

Wordlists

Determining which pages exist relies on a wordlist of commonly used directory and page names — conceptually similar to a password dictionary attack (covered later in the module). This approach won't reveal every page on a site (some use random or unique names), but it generally recovers the majority of pages —

reportedly up to a **90% success rate** on some websites.

Rather than building wordlists from scratch, the module points to the **GitHub SecLists repository**, which categorises wordlists by fuzzing type — including commonly used passwords, which get reused later for password brute-forcing.

On the PwnBox, the full SecLists repo is available at `/opt/useful/SecLists`. For page and directory fuzzing specifically, the module uses the commonly referenced `directory-list-2.3` wordlist family, available in several sizes:

```
xsystem32x@htb[/htb]$ locate directory-list-2.3-small.txt

/opt/useful/seclists/Discovery/Web-Content/directory-list-2.3-small.txt
```

Tip: this wordlist file starts with copyright comment lines, which count as part of the wordlist and add clutter to fuzzing results. Ffuf's **`-ic`** flag filters these out.

Section 3 — Directory Fuzzing

With the concept of web fuzzing and a wordlist in hand, this section puts ffuf to work finding actual website directories.

Installing and exploring ffuf

Ffuf comes pre-installed on the PwnBox. On a personal machine it can be installed with `apt install ffuf -y`, or downloaded directly from its GitHub repo. Running `ffuf -h` lists its options — the module highlights the ones relevant here:

Flag	Category	Purpose
-H	HTTP	Set a header "Name: Value"; multiple -H flags accepted
-X	HTTP	HTTP method to use (default: GET)
-b	HTTP	Cookie data, for copy-as-curl use
-d	HTTP	POST data
-recursion	HTTP	Scan recursively (URL must end in FUZZ)
-u	HTTP	Target URL
-mc	Matcher	Match HTTP status codes (default: 200,204,301,302,307,401,403)
-ms	Matcher	Match HTTP response size
-fc	Filter	Filter out HTTP status codes from the response
-fs	Filter	Filter out responses by size
-w	Input	Wordlist path, optionally with :KEYWORD
-o	Output	Write output to a file

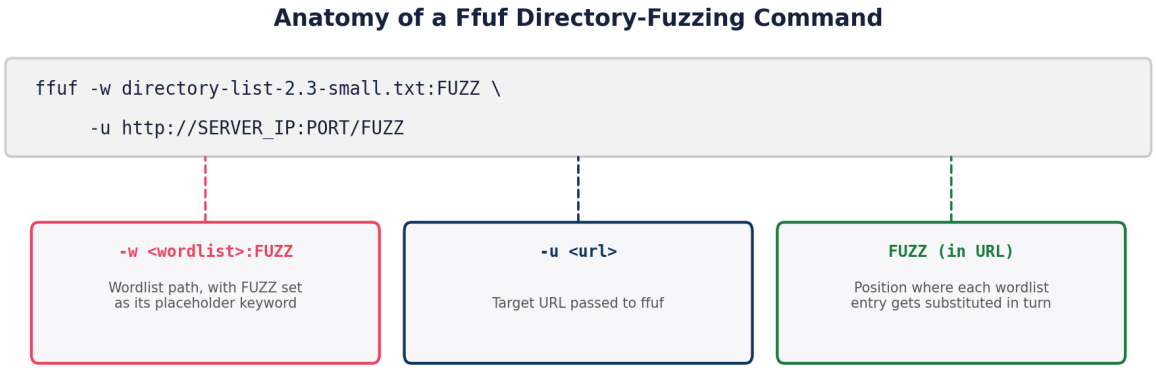
Building the fuzzing command

The two options that matter most here are `-w` for the wordlist and `-u` for the URL. A wordlist can be bound to a keyword by appending `:FUZZ` to its path:

```
xsystem32x@htb[/htb]$ ffuf -w
/opt/useful/seclists/Discovery/Web-Content/directory-list-2.3-small.txt:FUZZ
```

That FUZZ keyword then gets placed in the URL exactly where the directory name should go:

```
xsystem32x@htb[/htb]$ ffuf -w <SNIP> -u http://SERVER_IP:PORT/FUZZ
```



Running it

```
xsystem32x@htb[/htb]$ ffuf -w
/opt/useful/seclists/Discovery/Web-Content/directory-list-2.3-small.txt:FUZZ -u
http://SERVER_IP:PORT/FUZZ

...
blog [Status: 301, Size: 326, Words: 20, Lines: 10]
:: Progress: [87651/87651] :: Job [1/1] :: 9739 req/sec :: Duration: [0:00:09] :: Errors: 0
::
```

Here ffuf tested almost 90,000 URLs in under 10 seconds — speed varies with connection and ping, but it's extremely fast regardless. Thread count can be pushed higher (e.g. `-t 200`) to go even faster, but this isn't recommended against a remote target: it risks causing a Denial of Service on the target, or overwhelming the tester's own connection in severe cases.

The scan returned a hit for `blog`. Visiting `http://SERVER_IP:PORT/blog` directly returns an empty page — but critically, **not** a 404 or 403, confirming the directory exists and is accessible even though it has no dedicated content yet. The next section covers fuzzing for files and pages within a directory like this, to check whether it's genuinely empty or hiding something.

Section 3 lab question — answered

In addition to the directory found above, there is another directory that can be found. What is it?

Answer: forum

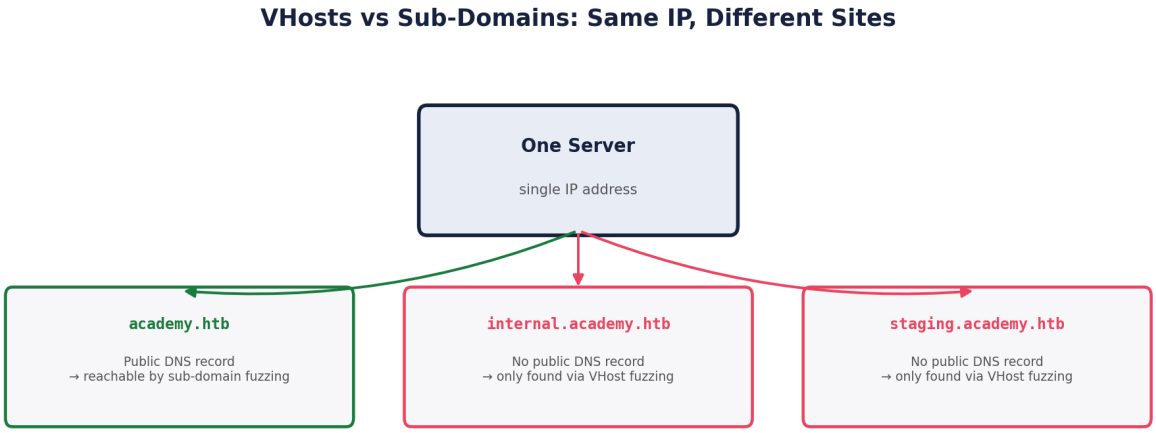
Running the same directory-fuzzing command against the target surfaces a second valid directory, `forum`, alongside the `blog` one already found — both returned a non-404/403 status, confirming they exist on the server.

Section 8 — Vhost Fuzzing

This section picks up from sub-domain fuzzing covered earlier in the module (Sections 4–7, not yet provided to build these notes). Sub-domain fuzzing using public DNS records works only for sub-domains that are actually published publicly. For sub-domains with no public DNS record, or for sites that aren't public at all, a different technique is needed: **Vhost Fuzzing**.

VHosts vs sub-domains

The key distinction: a VHost (virtual host) is effectively a "sub-domain" served from the **same server and the same IP** — a single IP can serve two or more entirely different websites. Critically, VHosts may or may not have a public DNS record.

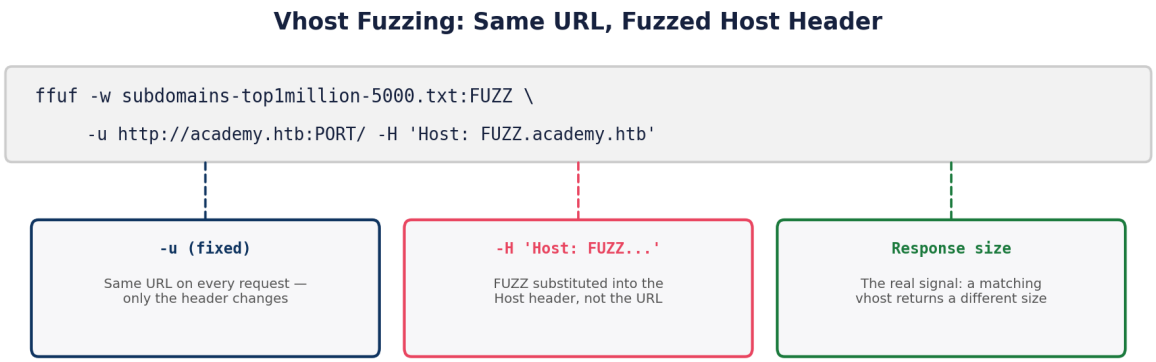


Many sites have non-public sub-domains that are deliberately left out of public DNS records — visiting them in a browser fails to connect, because public DNS simply has no record of their IP. Sub-domain fuzzing alone only ever finds the public ones. VHost fuzzing, run directly against an IP already known, can identify **both** public and non-public sub-domains and VHosts.

How VHost fuzzing works

Rather than manually adding every wordlist entry to `/etc/hosts`, VHost fuzzing works by fuzzing the HTTP `Host:` header itself, using ffuf's `-H` flag with the FUZZ keyword inside it:

```
xsystem32x@htb[/htb]$ ffuf -w
/opt/useful/seclists/Discovery/DNS/subdomains-top1million-5000.txt:FUZZ -u
http://academy.htb:PORT/ -H 'Host: FUZZ.academy.htb'
```



Reading the results — why status code alone isn't enough

Running this command, every single word in the wordlist comes back `200 OK`. This is expected, not a sign of success: the request is always hitting `http://academy.htb:PORT/`, only the `Host` header changes, and the server will answer `200` regardless of whether a matching VHost actually exists.

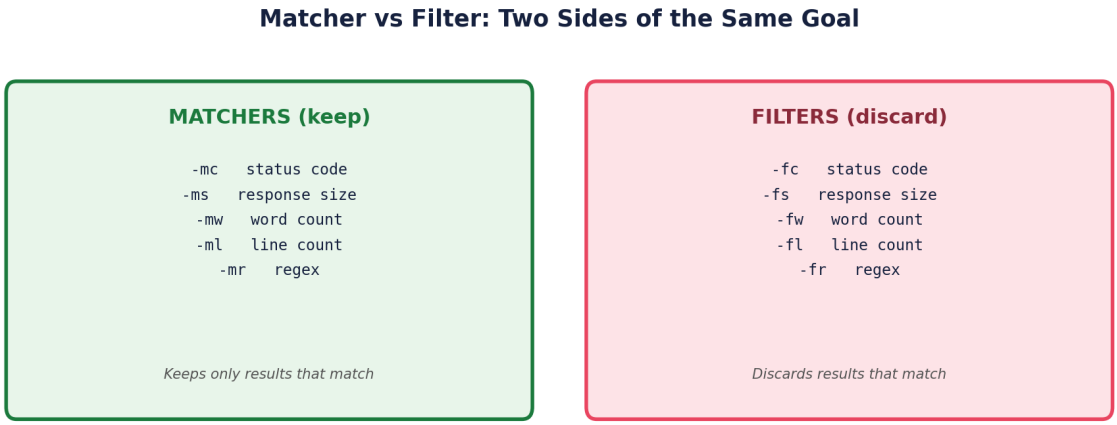
The real signal is **response size**. If a VHost genuinely exists and the correct one is sent in the header, the server serves that VHost's actual page — which is likely to differ in size from the generic / default response the other guesses receive. In other words, filtering or comparing by response size (not status code) is what separates real VHosts from noise in this technique.

Section 9 — Filtering Results

Up to now, no explicit filtering has been applied — ffuf's default matcher already filters out `404 Not Found` and keeps everything else. But as the VHost scan showed, that default isn't enough on its own: plenty of results can still come back as `200`, and a different factor is needed to separate real hits from noise.

Matching vs filtering

Ffuf can match or filter results on an HTTP status code, response size, or word count — surfaced by `ffuf -h`:



Flag	Type	Purpose
-mc	Matcher	Match HTTP status codes, or "all" for everything (default)
-ml	Matcher	Match amount of lines in response
-mr	Matcher	Match a regular expression
-ms	Matcher	Match HTTP response size
-mw	Matcher	Match amount of words in response
-fc	Filter	Filter HTTP status codes; comma-separated codes/ranges
-fl	Filter	Filter by line count; comma-separated counts/ranges
-fr	Filter	Filter by regular expression
-fs	Filter	Filter HTTP response size; comma-separated sizes/ranges
-fw	Filter	Filter by word count; comma-separated counts/ranges

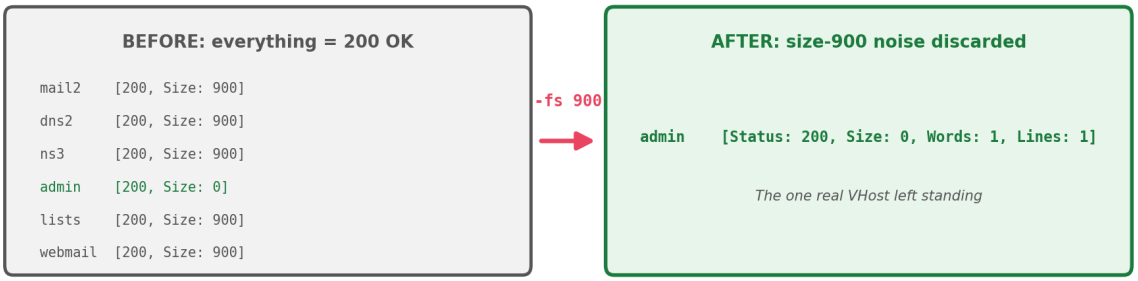
Applying it to the VHost scan

Matching isn't an option here, since the response size of a genuine (as-yet-unknown) VHost isn't known in advance. What *is* known is the size of the incorrect, noise results — 900 bytes, from the earlier test — so that can be filtered out directly with **`-fs 900`**:

```
xsystem32x@htb[/htb]$ ffuf -w
/opt/useful/seclists/Discovery/DNS/subdomains-top1million-5000.txt:FUZZ -u
http://academy.htb:PORT/ -H 'Host: FUZZ.academy.htb' -fs 900

...
admin [Status: 200, Size: 0, Words: 1, Lines: 1]
:: Progress: [4997/4997] :: Job [1/1] :: 1249 req/sec :: Duration: [0:00:04] :: Errors: 0
::
```

Filtering Out the Noise: -fs 900



With the 900-byte noise filtered out, a single real result remains: `admin`. Adding `admin.academy.htb` to `/etc/hosts` and visiting `https://admin.academy.htb:PORT/` confirms it — an accessible but empty page, different from the response at `academy.htb`, confirming this is genuinely a separate VHost. Visiting `https://admin.academy.htb:PORT/blog/index.php` returns a 404, further confirming this VHost has its own separate directory structure rather than sharing one with the main site.

Note: if the exercise instance restarts, double-check the port is still correct before revisiting the site. The section also sets up a follow-on exercise: running a recursive scan against `admin.academy.htb` to see what pages can be identified within it.